

INFOSYS SPRINGBOARD INTERNSHIP · TEAM 5

Intelligent Cognitive Alarm Platform

Milestone 3 Report

Adaptive Difficulty Engine & Habit Scoring Engine

Prepared by: Komala from team 5

Software Engineering Report — Backend Modules

Table of Contents

1. Cover Page.....	1
2. Milestone 3 Introduction	3
3. Milestone 3 Objectives	3
4. Adaptive Difficulty Engine.....	3
4.1 Purpose	3
4.2 Architecture	4
4.3 Workflow.....	4
4.4 Business Logic	5
4.5 Database Design	5
4.6 API Endpoints	5
4.7 Swagger Output.....	6
4.8 Advantages	7
4.9 Future Improvements.....	7
5. Habit Scoring Engine.....	7
5.1 Purpose	8
5.2 Architecture	8
5.3 Workflow.....	8
5.4 Weighted Formula	8
5.5 Grade Calculation.....	9
5.6 Database Design	9
5.7 API Endpoints	9
5.8 Swagger Output.....	10
5.9 Advantages & Future Improvements.....	11
6. Integrated Workflow	11
7. Database Tables	12
8. API Documentation	13
9. Testing.....	13
10. Results	14
11. Future Scope.....	14
12. Conclusion	15

2. Milestone 3 Introduction

The Intelligent Cognitive Alarm Platform is an AI-powered system that helps users build consistent wake-up habits by requiring them to solve personalized cognitive challenges before an alarm can be dismissed. Milestone 1 established the User Profile & Habit Management module, and Milestone 2 delivered the Wake-Up Verification and Challenge Evaluation modules that validate user answers and record performance on each challenge.

Milestone 3 builds on this foundation by introducing adaptive intelligence: the platform now analyzes a user's historical performance to automatically adjust challenge difficulty, and consolidates wake-up behaviour into a single, interpretable habit score. Together, these two modules move the platform from simply recording user activity to actively reasoning about it and personalizing the experience in response.

This report documents the two modules delivered in Milestone 3 — the Adaptive Difficulty Engine and the Habit Scoring Engine — covering their purpose, architecture, workflow, business logic, database design, API implementation, and testing, along with the integrated workflow that connects them to the modules delivered in earlier milestones.

3. Milestone 3 Objectives

The primary objective of Milestone 3 is to implement adaptive intelligence features that analyze user performance and improve the wake-up experience over time. Specifically:

- Automatically adjust challenge difficulty based on a user's recent success rate, so the platform neither bores a high performer nor discourages a struggling one.
- Calculate a single, weighted habit score that summarizes wake-up consistency, challenge performance, snooze behaviour, and sleep-schedule adherence.
- Translate the habit score into a qualitative grade that is easy for users to interpret at a glance.
- Persist both difficulty and habit-score history in PostgreSQL so that later milestones (recommendation and analytics features) can build on this data.
- Expose both modules through a consistent, fully tested FastAPI/Swagger interface, matching the pattern established in Milestones 1 and 2.

4. Adaptive Difficulty Engine

The Adaptive Difficulty Engine is the module responsible for deciding how hard the user's next cognitive challenge should be. Rather than presenting a fixed difficulty for every user, the engine reads each user's recent performance — captured by the Challenge Evaluation module in Milestone 2 — and computes an appropriate next difficulty level.

4.1 Purpose

The purpose of the Adaptive Difficulty Engine is to automatically adjust challenge difficulty according to the user's previous performance. This keeps the wake-up challenge appropriately demanding: users who are consistently succeeding are moved to harder challenges that require more

cognitive engagement, while users who are struggling are moved to easier challenges so the alarm remains dismissible without becoming a source of frustration.

4.2 Architecture

The module follows the same layered architecture used throughout the platform: a FastAPI router handles HTTP requests, a service layer contains the difficulty-calculation business logic, and SQLAlchemy ORM persists results to PostgreSQL.

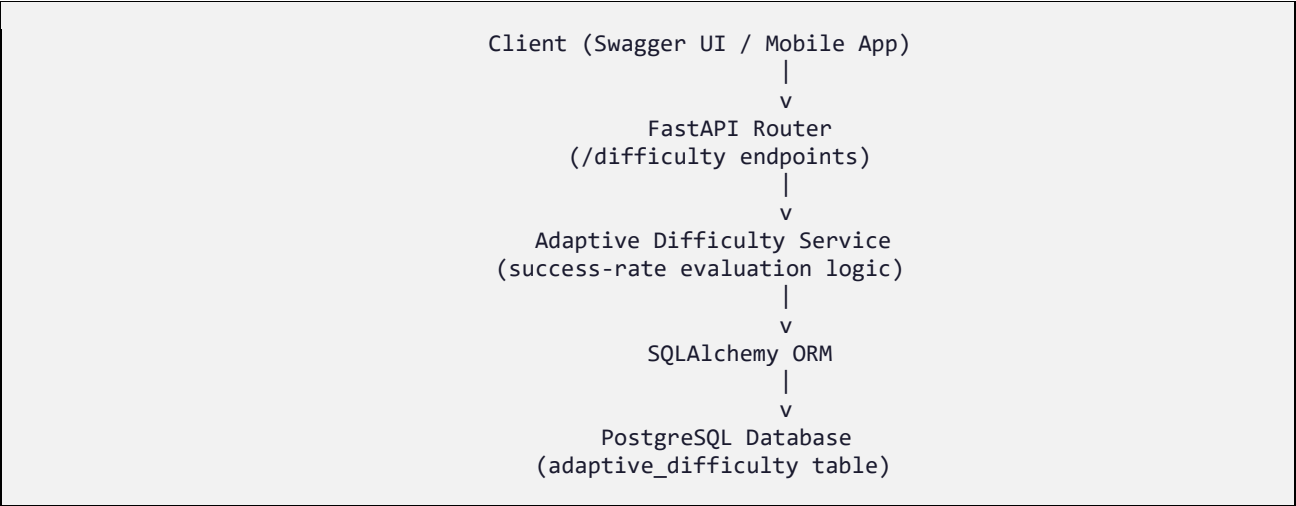


Figure 1: Layered architecture of the Adaptive Difficulty Engine.

4.3 Workflow

The engine is triggered once a user's performance data is available from the Challenge Evaluation module. The end-to-end workflow is as follows:

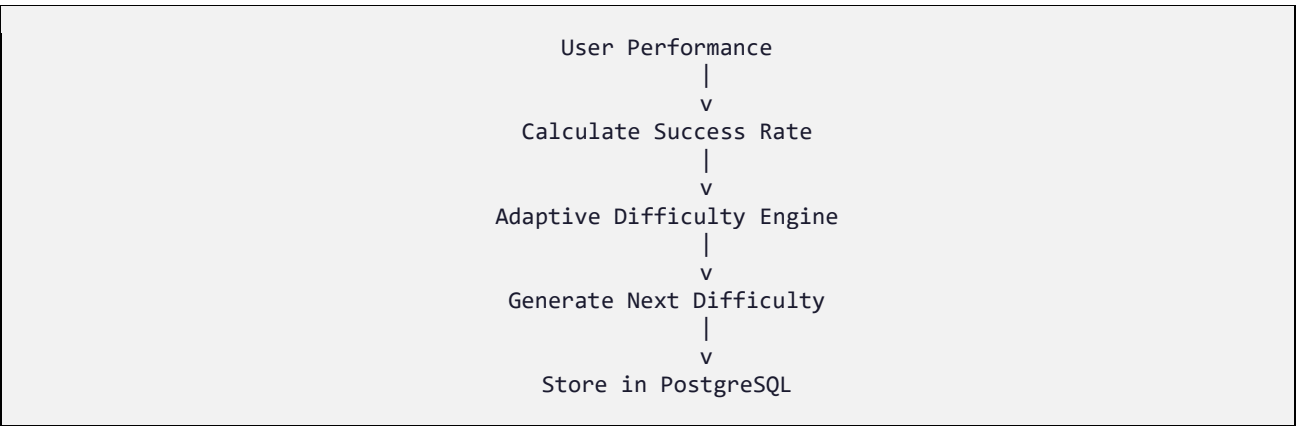
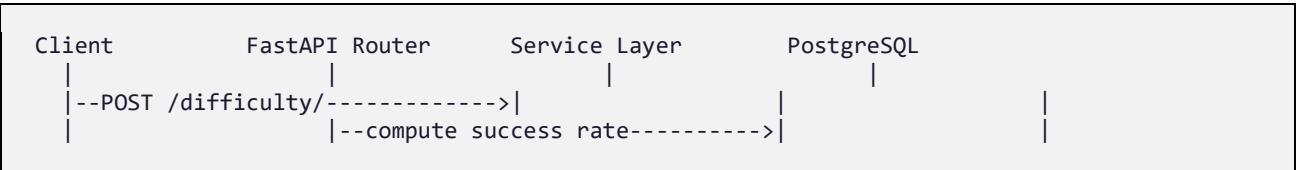


Figure 2: Adaptive Difficulty Engine workflow.

As a sequence of interactions between the client, the API layer, and the database, the same flow can be expressed as:



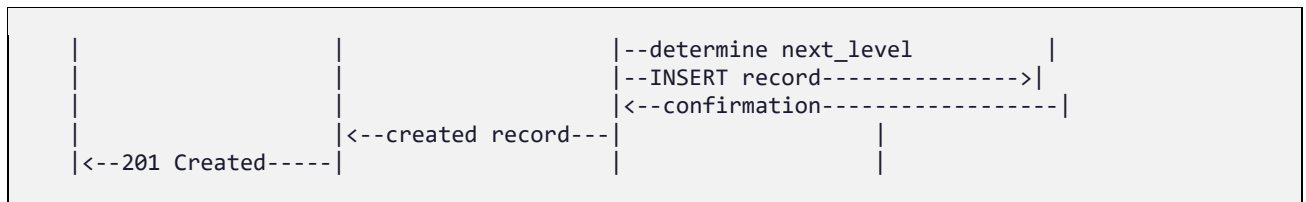


Figure 3: Sequence diagram for POST /difficulty/.

4.4 Business Logic

The core decision rule of the Adaptive Difficulty Engine is a threshold-based classifier applied to the user's success rate, calculated from their recent challenge attempts recorded by the Challenge Evaluation module:

Condition	Next Difficulty
Success Rate $\geq$ 80%	Hard
50% $\leq$ Success Rate $<$ 80%	Medium
Success Rate $<$ 50%	Easy

This three-tier rule keeps the logic simple, deterministic, and easy to reason about for both developers and evaluators, while still being responsive to real changes in user performance. The `current_level` field records the difficulty the user is on now, and `next_level` records the engine's computed recommendation for the following challenge, so that both current state and the adaptive decision are auditable from a single record.

4.5 Database Design

The `adaptive_difficulty` table stores one record per difficulty evaluation, linked to the user it was computed for:

Field	Type	Description
<code>id</code>	integer	Primary key, unique record identifier
<code>user_id</code>	integer	Foreign key referencing the user
<code>success_rate</code>	float	User's success rate on prior challenges (%)
<code>current_level</code>	string	Difficulty level the user is currently on
<code>next_level</code>	string	Difficulty level computed for the next challenge

4.6 API Endpoints

The Adaptive Difficulty Engine exposes a standard set of REST endpoints, consistent with the CRUD pattern used across the platform:

Method	Path	Description
--------	------	-------------

POST	/difficulty/	Create a new adaptive difficulty record for a user
GET	/difficulty/	Retrieve all adaptive difficulty records
GET	/difficulty/{id}	Retrieve a single adaptive difficulty record by ID
DELETE	/difficulty/{id}	Delete an adaptive difficulty record

4.7 Swagger Output

The endpoints below were implemented in FastAPI and verified against the auto-generated Swagger UI (/docs), following the same testing approach used in Milestones 1 and 2.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/adaptive/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": 1,
    "success_rate": 82,
    "current_level": "Medium"
  }'
```

Request URL

http://127.0.0.1:8000/adaptive/

Server response

Code

Details

200

Response body

```
{
  "id": 2,
  "success_rate": 82,
  "current_level": "Medium",
  "next_level": "Hard",
  "user_id": 1
}
```

Response headers

```
content-length: 85
content-type: application/json
date: Sat, 08 Aug 2026 12:58:32 GMT
server: uvicorn
```

request url

http://127.0.0.1:8000/adaptive/

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "success_rate": 82,
    "current_level": "Medium",
    "next_level": "Hard",
    "user_id": 1
  },
  {
    "id": 2,
    "success_rate": 82,
    "current_level": "Medium",
    "next_level": "Hard",
    "user_id": 1
  }
]
```

Response headers

```
content-length: 173
content-type: application/json
date: Sat, 08 Aug 2026 12:58:41 GMT
server: uvicorn
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "id": 1,
  "success_rate": 82,
  "current_level": "Medium",
  "next_level": "Hard",
  "user_id": 1
}
```

Response headers

```
content-length: 85
content-type: application/json
date: Sat,08 Aug 2026 13:04:42 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200	
-----	--

Successful Response

Code

Details

200	
-----	--

Response body

```
{
  "id": 1,
  "success_rate": 82,
  "current_level": "Medium",
  "next_level": "Hard",
  "user_id": 1
}
```

Response headers

```
content-length: 85
content-type: application/json
date: Sat,08 Aug 2026 13:05:14 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200	
-----	--

Successful Response

4.8 Advantages

- Keeps challenges appropriately difficult for each individual user, rather than applying a one-size-fits-all difficulty.
- Simple, threshold-based logic that is easy to test, verify, and explain to non-technical stakeholders.
- Stores both current and next difficulty, giving downstream modules (and the user interface) full visibility into the adaptation decision.
- Decoupled from the Challenge Evaluation module through a clean service boundary, so either module can evolve independently.

4.9 Future Improvements

- Replace the fixed 50%/80% thresholds with a machine-learning model trained on longer-term user performance trends.
- Incorporate additional signals such as time-of-day performance and challenge type into the difficulty decision.
- Add a smoothing mechanism so difficulty does not oscillate sharply after a single unusually good or bad attempt.

5. Habit Scoring Engine

The Habit Scoring Engine consolidates a user's wake-up behaviour into a single, weighted score and an associated grade. Where the Adaptive Difficulty Engine reacts to performance in the moment, the Habit Scoring Engine provides a longer-term, holistic view of how well a user is building consistent wake-up habits.

5.1 Purpose

The purpose of the Habit Scoring Engine is to calculate an overall habit score based on the user's wake-up behaviour, combining multiple weighted signals into one number and one grade that is simple for the user to understand and track over time.

5.2 Architecture

The Habit Scoring Engine follows the same layered architecture as the rest of the platform, keeping the scoring logic isolated in its own service layer:

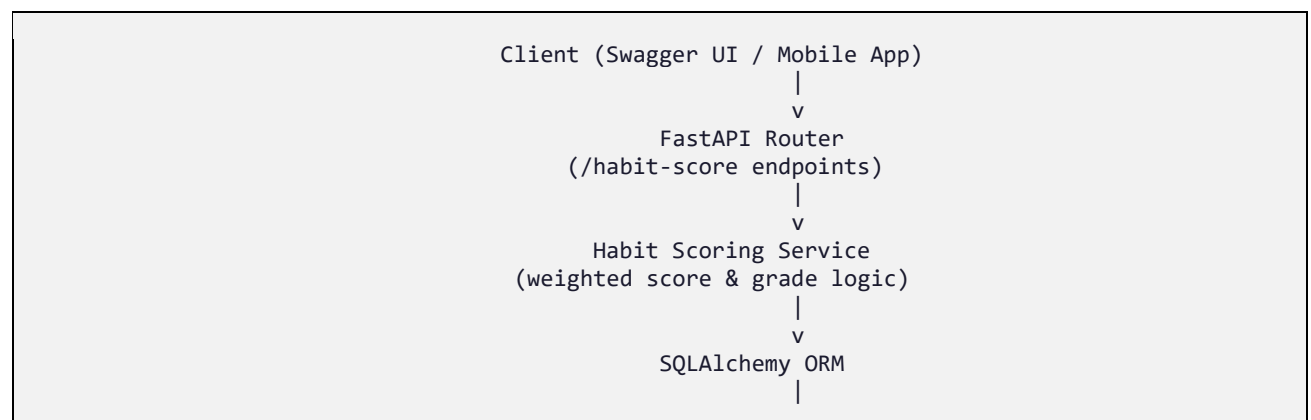




Figure 8: Layered architecture of the Habit Scoring Engine.

5.3 Workflow

The engine consumes wake-up, challenge, snooze, and sleep data already captured by earlier modules, and produces one composite result:

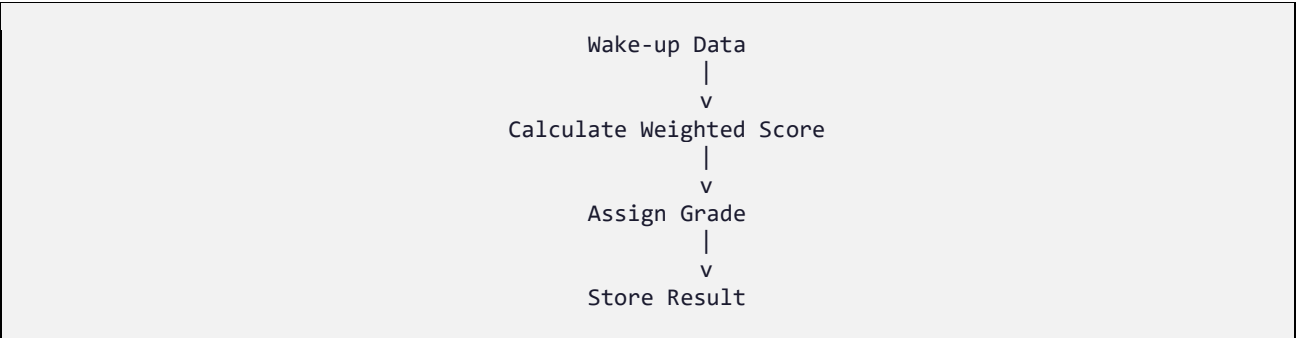


Figure 9: Habit Scoring Engine workflow.

5.4 Weighted Formula

The overall habit score is a weighted sum of four component scores, each contributing a fixed percentage to the final result:

Overall Score = (Wake-up Score x 0.35)
 + (Challenge Score x 0.25)
 + (Snooze Score x 0.20)
 + (Sleep Score x 0.20)

Figure 10: Weighted scoring formula used by the Habit Scoring Engine.

Wake-up consistency carries the largest weight (35%), reflecting that reliably waking at the intended time is the platform's central goal. Challenge performance (25%) reflects cognitive engagement during the wake-up process, while snooze behaviour and sleep-schedule adherence (20% each) capture secondary but still meaningful habit signals.

5.5 Grade Calculation

The numeric overall_score is translated into a qualitative grade using the following bands:

Score Range	Grade
80 – 100	Excellent
60 – 79	Good
40 – 59	Average
Below 40	Poor

Presenting a grade alongside the raw score gives users an at-a-glance read on their habit quality without requiring them to interpret a raw percentage.

5.6 Database Design

The `habit_score` table stores one record per scoring run, capturing each component score alongside the composite result:

Field	Type	Description
<code>id</code>	integer	Primary key, unique record identifier
<code>user_id</code>	integer	Foreign key referencing the user
<code>wakeup_score</code>	float	Score derived from wake-up consistency
<code>challenge_score</code>	float	Score derived from challenge completion
<code>snooze_score</code>	float	Score derived from snooze behaviour
<code>sleep_score</code>	float	Score derived from sleep schedule adherence
<code>overall_score</code>	float	Weighted composite score (0–100)
<code>grade</code>	string	Qualitative grade derived from <code>overall_score</code>

5.7 API Endpoints

The Habit Scoring Engine exposes the same standard CRUD pattern used by the Adaptive Difficulty Engine and earlier modules:

Method	Path	Description
POST	<code>/habit-score/</code>	Create a new habit score record for a user
GET	<code>/habit-score/</code>	Retrieve all habit score records
GET	<code>/habit-score/{id}</code>	Retrieve a single habit score record by ID
DELETE	<code>/habit-score/{id}</code>	Delete a habit score record

5.8 Swagger Output

The endpoints below were implemented in FastAPI and verified against the auto-generated Swagger UI (`/docs`):

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "wakeup_score": 90,
  "id": 2,
  "snooze_score": 70,
  "overall_score": 82.5,
  "challenge_score": 80,
  "user_id": 1,
  "sleep_score": 85,
  "grade": "Excellent"
}
```

Response headers

```
content-length: 143
content-type: application/json
date: Sat,08 Aug 2026 12:56:47 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200	Successful Response
-----	---------------------

200	
-----	--

Response body

```
[
  {
    "wakeup_score": 90,
    "id": 1,
    "snooze_score": 70,
    "overall_score": 82.5,
    "challenge_score": 80,
    "user_id": 1,
    "sleep_score": 85,
    "grade": "Excellent"
  },
  {
    "wakeup_score": 90,
    "id": 2,
    "snooze_score": 70,
    "overall_score": 82.5,
    "challenge_score": 80,
    "user_id": 1,
    "sleep_score": 85,
    "grade": "Excellent"
  }
]
```

Response headers

```
content-length: 289
content-type: application/json
date: Sat,08 Aug 2026 13:02:07 GMT
server: uvicorn
```

Responses

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "wakeup_score": 90,
  "id": 1,
  "snooze_score": 70,
  "overall_score": 82.5,
  "challenge_score": 80,
  "user_id": 1,
  "sleep_score": 85,
  "grade": "Excellent"
}
```

Response headers

```
content-length: 143
content-type: application/json
date: Sat,08 Aug 2026 13:03:15 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200	Successful Response
-----	---------------------

200	
-----	--

Response body

```
{
  "wakeup_score": 90,
  "id": 1,
  "snooze_score": 70,
  "overall_score": 82.5,
  "challenge_score": 80,
  "user_id": 1,
  "sleep_score": 85,
  "grade": "Excellent"
}
```

Response headers

```
content-length: 143
content-type: application/json
date: Sat,08 Aug 2026 13:03:59 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

200	Successful Response
-----	---------------------

5.9 Advantages & Future Improvements

Advantages

- Reduces four separate behavioural signals into one interpretable score and grade.
- Weighting is explicit and configurable, so the relative importance of each signal can be tuned without changing the underlying data model.
- Provides a stable metric that later modules — recommendations, dashboards, notifications — can build on directly.

Future Improvements

- Allow weights to be personalized per user rather than fixed platform-wide.
- Track score trends over time (weekly/monthly) rather than only the latest snapshot.
- Feed the habit score back into the Adaptive Difficulty Engine as an additional input signal.

6. Integrated Workflow

The Adaptive Difficulty Engine and Habit Scoring Engine do not operate in isolation — they sit at the end of a pipeline that begins with the modules delivered in Milestones 1 and 2. Wake-Up Verification confirms whether a user's answer to a challenge was correct, and Challenge Evaluation records the resulting score, attempts, and time taken. The Adaptive Difficulty Engine reads this evaluation data to decide the next challenge's difficulty, and the Habit Scoring Engine combines wake-up, challenge, snooze, and sleep data into the final habit score and grade.

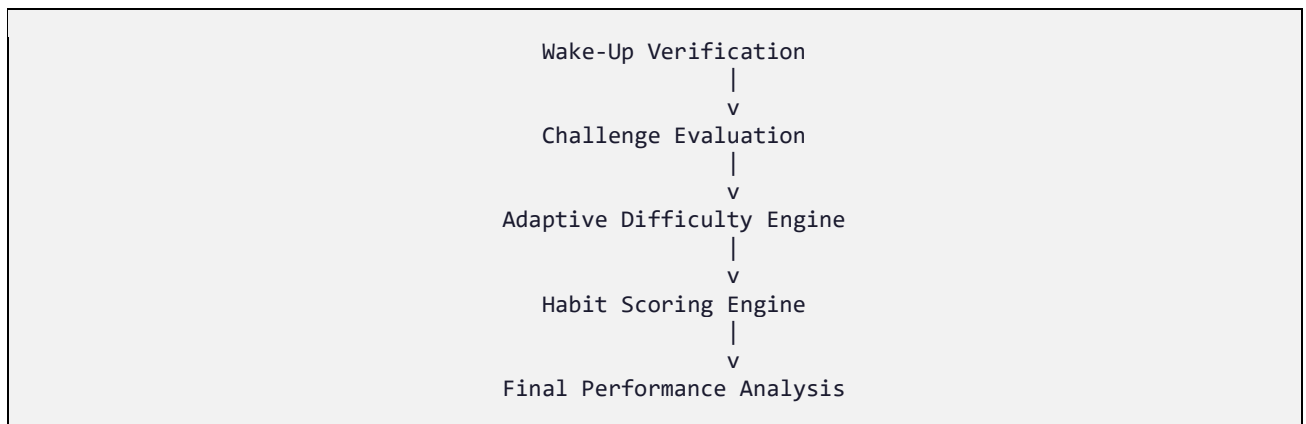


Figure 15: Integrated workflow across Milestones 1–3.

This integrated pipeline means each module's output becomes the next module's input: verification results feed evaluation, evaluation results feed difficulty adaptation and habit scoring, and the resulting habit score becomes the foundation for the recommendation and analytics features planned for later milestones.

7. Database Tables

The two Milestone 3 modules introduce two new PostgreSQL tables, summarized here for reference alongside their field definitions from Sections 4.5 and 5.6.

7.1 adaptive_difficulty

Field	Type	Description
id	integer	Primary key, unique record identifier
user_id	integer	Foreign key referencing the user
success_rate	float	User's success rate on prior challenges (%)
current_level	string	Difficulty level the user is currently on
next_level	string	Difficulty level computed for the next challenge

7.2 habit_score

Field	Type	Description
id	integer	Primary key, unique record identifier
user_id	integer	Foreign key referencing the user
wakeup_score	float	Score derived from wake-up consistency
challenge_score	float	Score derived from challenge completion
snooze_score	float	Score derived from snooze behaviour
sleep_score	float	Score derived from sleep schedule adherence
overall_score	float	Weighted composite score (0–100)
grade	string	Qualitative grade derived from overall_score

Both tables reference user_id as a foreign key back to the User Profile module established in Milestone 1, keeping user identity centralized while allowing each module to own its own behavioural data.

8. API Documentation

The complete set of endpoints delivered in Milestone 3 is summarized below.

8.1 Adaptive Difficulty Engine

Method	Path	Description
POST	/difficulty/	Create a new adaptive difficulty record for a user
GET	/difficulty/	Retrieve all adaptive difficulty records
GET	/difficulty/{id}	Retrieve a single adaptive difficulty record by ID
DELETE	/difficulty/{id}	Delete an adaptive difficulty record

8.2 Habit Scoring Engine

Method	Path	Description
POST	/habit-score/	Create a new habit score record for a user
GET	/habit-score/	Retrieve all habit score records
GET	/habit-score/{id}	Retrieve a single habit score record by ID
DELETE	/habit-score/{id}	Delete a habit score record

9. Testing

9.1 Swagger Testing

Every endpoint in both modules was exercised directly through the FastAPI Swagger UI (/docs), with live request payloads submitted and the resulting response codes and bodies inspected. This included create, retrieve (all and by ID), and delete operations for both `adaptive_difficulty` and `habit_score` records.

9.2 Database Testing

Alongside API-level testing, records were verified directly in PostgreSQL to confirm that computed values (`next_level`, `overall_score`, `grade`) were persisted correctly and matched the values returned by the API response.

9.3 Expected Outputs

- POST requests return HTTP 201 with the created record, including any server-computed fields (`next_level`, `overall_score`, `grade`).
- GET requests return HTTP 200 with either a list of records or a single record matching the given ID.
- DELETE requests return HTTP 200 with a confirmation message once the record has been removed.
- Business logic outputs (`next_level`, `grade`) match the documented thresholds for every tested input combination.

10. Results

Both the Adaptive Difficulty Engine and the Habit Scoring Engine were implemented, integrated with PostgreSQL, and verified end-to-end through the Swagger UI. All CRUD endpoints for both modules return the expected status codes and payloads, and the underlying business logic — the three-tier difficulty rule and the four-component weighted score — produces results consistent with the documented specification across the tested input ranges.

With these two modules complete, the platform now has a full pipeline from raw user answers through to a single, interpretable habit score, satisfying the Milestone 3 objective of implementing adaptive intelligence features that analyze user performance and improve the wake-up experience.

11. Future Scope

Building on the adaptive intelligence foundation delivered in Milestone 3, the following enhancements are planned for future milestones:

- AI Recommendation Engine — surfacing personalized suggestions (bedtime adjustments, challenge types) based on habit score trends.
- Machine Learning based Difficulty Prediction — replacing the fixed threshold rule with a trained model that adapts to individual learning curves.
- Behavior Analytics — deeper analysis of snooze patterns, wake-up timing, and engagement trends over weeks and months.
- Android Mobile Application Integration — surfacing difficulty and habit-score data natively within the React Native mobile app.
- Dashboard Analytics — a Wellness Coach / Admin-facing dashboard visualizing habit scores and difficulty trends across users.

12. Conclusion

Milestone 3 delivers the Adaptive Difficulty Engine and Habit Scoring Engine, extending the Intelligent Cognitive Alarm Platform from a system that records wake-up activity into one that actively adapts to it. The Adaptive Difficulty Engine keeps each user's challenges appropriately calibrated using a transparent, threshold-based rule, while the Habit Scoring Engine distills wake-up, challenge, snooze, and sleep behaviour into a single weighted score and grade. Both modules follow the same FastAPI/PostgreSQL/SQLAlchemy architecture established in earlier milestones, are fully documented through Swagger, and are positioned to feed directly into the recommendation, analytics, and mobile-integration features planned for subsequent milestones.